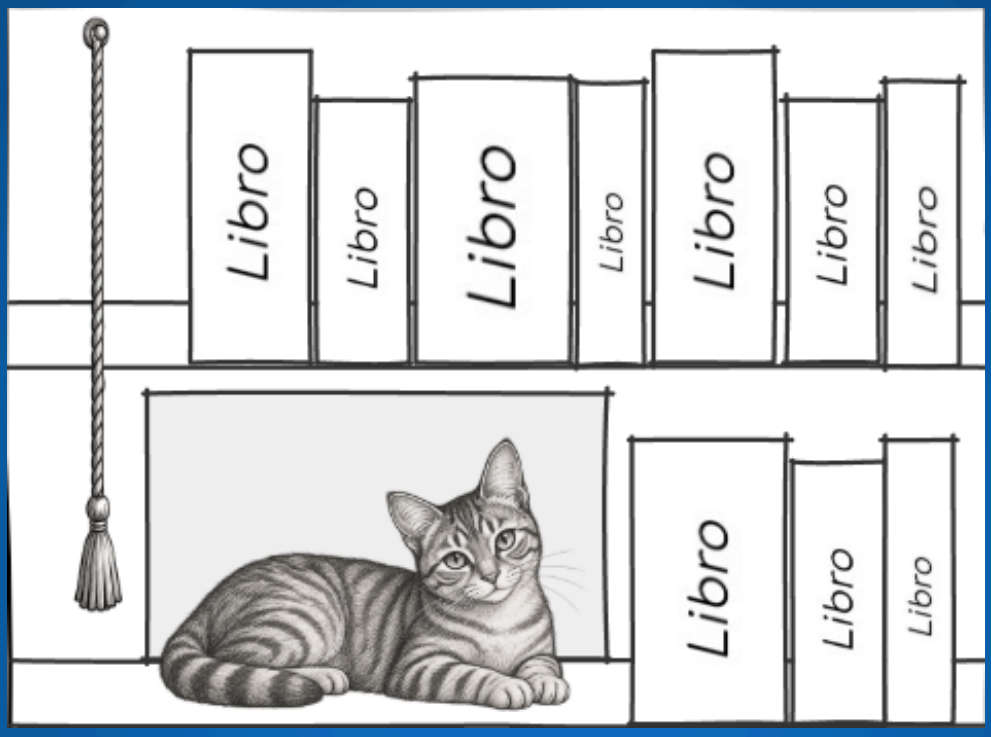
Nook & Ink — Entrega 1.1: lista y detalle con Jetpack Compose



IES VIRGEN DEL CARMEN

CFGS DE DAM

Módulo:	Programación Multimedia y Dispositivos Móviles
Nombre:	Manuel Fuentes Cruz
Curso y grupo:	2º DAM B
Profesor:	Matías Montávez Sánchez

ÍNDICE DE CONTENIDOS

1. RESUMEN E IDEA....3
2. INTRODUCCIÓN Y JUSTIFICACIÓN....3
3. OBJETIVOS DEL TRABAJO....5
4. DESARROLLO / CUERPO DEL TRABAJO....5
4.1. Estudio previo y organización....5
4.1.1 Libro.kt.... 6
4.1.2 DatosLibros.kt.... 6
4.1.3 MainActivity.kt.... 7
4.1.4 AppPantallas.kt.... 7
4.1.5 PantallaEstanteria.kt.... 8
4.1.6 PantallaDetalle.kt.... 9
4.2 Capturas.... 10
5. CONCLUSIONES Y VALORACIÓN PERSONAL.... 13
6. COSAS QUE NO HEMOS VISTO....14
7. ANEXOS....15

1. RESUMEN E IDEA

En esta entrega 1.1 hay que hacer dos pantallas con Jetpack Compose: una lista con elementos de ejemplo y un detalle al que se llega pulsando uno de ellos y desde el que se puede volver. En la APP la lista es una estantería de libros y el detalle muestra el libro que elijas.

He resuelto las dos pantallas con una única Activity y gestionando el estado en Compose, como explican los apuntes en el apartado 15.3 de la UD2. Cada pantalla es una función @Composable en su propio archivo y una función central, AppPantallas, decide cuál se ve. Todavía no he hecho nada de la navegación formal de la 1.4, pero sí he investigado y consultado los apuntes para dejar la estructura preparada.

2. INTRODUCCIÓN Y JUSTIFICACIÓN

Como expliqué en la 1.0, mi aplicación será una biblioteca o estantería personal con algunas funciones extra. En esta entrega se piden varias cosas que he tenido que adaptar a esa temática, y las detallo en el desarrollo.

Las dos pantallas que he creado son:

• Estantería: la pantalla principal, donde se ven los libros que el usuario quiere mostrar, con sus distintos colores y decoraciones (por ahora son imágenes de IA generativa).

• Detalle: se abre al pulsar un libro y enseña la información de ese libro: el título completo, la valoración en estrellas y la descripción. Tiene un botón X para volver a la estantería.

He tomado tres decisiones importantes:

1. Una Activity y estado en Compose. Al principio pensé en una Activity por pantalla comunicadas con Intent, pero los apuntes no explican cómo pasar datos entre Activities y el enunciado dice que basta con gestionar el estado de la interfaz. Por eso uso la alternativa del apartado 15.3 de la UD2. Además, la UD3 también usa una sola Activity, así que en la 1.4 solo tendré que sustituir la elección de pantalla por la navegación.

2. Una tarjeta por balda, no por libro. El enunciado pide que cada elemento sea una tarjeta de Material. En mi prototipo la estantería es el elemento visual y los libros se apoyan en ella, así que cada balda es un ElevatedCard y cada libro es lo que se pulsa.

3. Datos temporales. Los 18 libros están escritos en el código. No se crean, modifican, eliminan ni guardan, tal y como indica el enunciado.

3. OBJETIVOS DEL TRABAJO

- Comenzar a crear la APP.
- Crear las dos primeras pantallas con su estructura: una lista con datos de ejemplo y un detalle de cada elemento.
- Conectar las pantallas gestionando el estado en Compose, de forma que se pueda abrir el detalle de un libro y volver a la lista.
- Dejar una estructura clara, con una función @Composable por pantalla, que se pueda reutilizar en la 1.4

4. DESARROLLO / CUERPO DEL TRABAJO

4.1. Estudio previo y organización

Para entender cómo funcionan las pantallas hay que tener en cuenta que la Activity es la misma durante todo el proceso, pero el contenido que muestra puede cambiar. Cuando se pulsa un libro de la estantería, esta no se encarga de mostrar su detalle, sino que simplemente avisa de qué libro se ha pulsado. Quien gestiona ese cambio es AppPantallas, que guarda el id del libro seleccionado, lo utiliza para buscar sus datos y muestra su pantalla de detalle. Al pulsar la X, se borra ese id y vuelve a mostrarse la estantería.

Cada pantalla es una función @Composable en su propio archivo. El proyecto queda así:

```
app/src/main/java/com/example/appbiblioteca/
├── MainActivity.kt
├── AppPantallas.kt
├── PantallaEstanteria.kt
├── PantallaDetalle.kt
├── DatosLibros.kt
└── model/
    └── Libro.kt
```

4.1.1 Libro.kt

Para empezar, tenemos la data class que guarda los datos de un libro. Cada libro tiene un id, un título, un título corto para el lomo, una descripción, un color y una valoración. El resto de campos tienen un valor por defecto.

El color (colorLomo) es directamente un Color de Compose, que se escribe como Color(0xFF8E3B2F).

La altura del lomo (altoEnDp) toma un valor aleatorio entre 85 y 130 dp, para que cada libro tenga una altura distinta, como en una estantería real. Por el momento me parece suficiente con la altura: también se podría variar el ancho, pero sería más difícil pulsar los libros más pequeños. La altura cambia cada vez que se reinicia la aplicación, pero se mantiene al girar el móvil mientras la app sigue abierta.

Los campos tieneImagen, imagen y mostradoEnEstanteria todavía no se usan. Son variables que dejo preparadas para entregas más avanzadas.

4.1.2 DatosLibros.kt

La lista listaDeLibros contiene los 18 libros de ejemplo. Es pública porque la usan varios archivos: AppPantallas, para buscar el libro seleccionado, y PantallaEstanteria, para dibujarlos.

La función crearBaldas reparte los libros por los niveles de la estantería, que son las baldas. Recorre los libros de uno en uno y los va metiendo en una balda; cuando la balda se llena, la guarda y abre otra. La primera balda lleva 6 libros, la segunda 3, y así alternando 6 y 3, porque las baldas de 3 libros dejan sitio a una decoración que ocupa el espacio de otros 3. He puesto 18 libros para que salgan 4 baldas justas (6 + 3 + 6 + 3). Si sobraran libros, la línea justo antes del return guarda también la última balda incompleta.

El resultado se guarda en listaDeBaldas, que también es pública porque la usa PantallaEstanteria.

4.1.3 MainActivity.kt

MainActivity es la única Activity de la app. Su método onCreate se ejecuta al abrirse la pantalla, setContent indica que la interfaz se dibuja con Compose y AppBibliotecaTheme aplica los colores y estilos de la app. Dentro solo llama a AppPantallas(), que decide qué pantalla se ve.

4.1.4 AppPantallas.kt

AppPantallas guarda en una variable de estado qué libro está seleccionado: var idSeleccionado by rememberSaveable { mutableStateOf<String?>(null) }. Si vale null no hay ningún libro seleccionado y se ve la estantería; si tiene un id, se ve el detalle de ese libro. Guardo solo el id, no el libro entero, porque basta para buscarlo en la lista con listaDeLibros.find { it.id == idSeleccionado }. Uso rememberSaveable y no remember para que, al girar el móvil, no se vuelva a la estantería.

Las pantallas no se conocen entre sí: solo avisan con funciones. PantallaEstanteria recibe onLibroClick y, cuando se pulsa un libro, su lambda guarda el id de ese libro. PantallaDetalle recibe onVolver y su lambda vuelve a poner el estado en null. El ciclo es el de los apuntes: el usuario pulsa, se ejecuta el evento, la lambda cambia el estado y Compose dibuja la pantalla nueva.

Aquí también guardo el estado de la lista (rememberLazyListState) y se lo paso a la estantería. Cuando se muestra el detalle, la lista desaparece y perdería su posición, así que al volver saltaría al principio. Guardándola en AppPantallas, que no desaparece, la estantería sigue en el mismo punto.

Por último BackHandler sirve para que el boton de atras del movil cierre el detalle y vaya a la estantería, sin eso, directamente se cerraría la APP al ir hacia atrás

4.1.5 PantallaEstanteria.kt

PantallaEstanteria es la pantalla principal. Recibe onLibroClick, una lambda que significa «cuando pulses un libro, avísame»: la pantalla no decide nada, solo avisa. También recibe el estado de la lista que viene de AppPantallas.

Dentro hay un Scaffold con su innerPadding (apartado 5.1 de los apuntes), que deja el contenido dentro de la zona libre de la pantalla para que no lo tapen las barras del sistema. Después va un LazyColumn, una lista vertical con scroll que solo dibuja lo que se ve en ese momento, así gasta menos memoria aunque haya muchos elementos. Primero va un item con el título de la app (NOOK & INK) y después un itemsIndexed, que crea una tarjeta por cada balda y da además el número de la balda, que sirve para decidir de qué lado va la decoración.

TarjetaDeBalda es un ElevatedCard, una tarjeta con sombra. En mi aplicación no cuadraba hacer una tarjeta por cada libro y tenerlos así en la estantería, así que cada nivel del LazyColumn es un ElevatedCard con la fila de libros dentro. Me ha gustado cómo ha quedado. Dentro hay un Column con el color de fondo de la balda, que se aplica antes del padding para que cubra también el margen. Arriba va la fila de libros y debajo la tabla de madera, que es un Box de 8 dp de alto con color madera.

La fila (Row) mide 145 dp de alto y tiene el verticalAlignment abajo, así que los libros se apoyan en la tabla de madera. El weight reparte el ancho en proporciones: cada libro pesa 1 y la decoración pesa 3. En una balda de 6 libros cada uno ocupa 1/6, y en una de 3 libros más la decoración también suman 6 partes, así que los libros tienen el mismo ancho en todas las baldas.

La variable decoracionALaIzquierda hace que la decoración alterne de lado. Como solo hay decoración en las baldas pares, la balda 2 la lleva a la izquierda, la 4 a la derecha, la 6 a la izquierda, y así. Lo he hecho siempre igual para no añadir aleatoriedad.

LomoDeLibro es un Box con su altura propia, que se puede pulsar (clickable), con el color del libro de fondo y un borde. Al pulsarlo llama a onClick, que sube hasta onLibroClick. Dentro va el título del lomo en blanco, girado 90 grados con rotate(90f) para que se lea en vertical. WrapContentWidth(unbounded = true) permite que el texto sea más largo que el ancho del lomo sin recortarse.

Decoracion muestra la imagen de decoración con Image. Lleva contentDescription, un texto que se usa para accesibilidad y que puede leer un lector de pantalla. Uso ContentScale.Fit para que la imagen se vea entera y no se recorte al girar el móvil. La idea de la app es que se pueda pulsar el hueco de la decoración y elegir otra, pero eso será otra pantalla y lo haré en otra entrega.

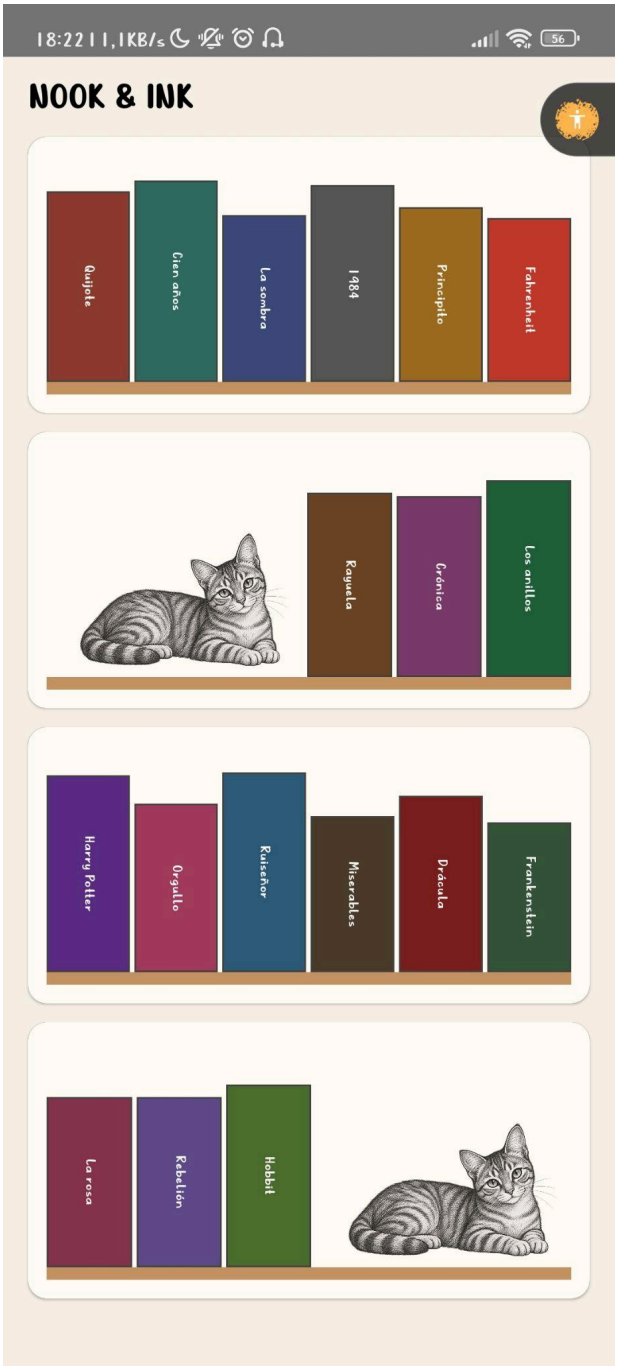
4.1.6 PantallaDetalle.kt

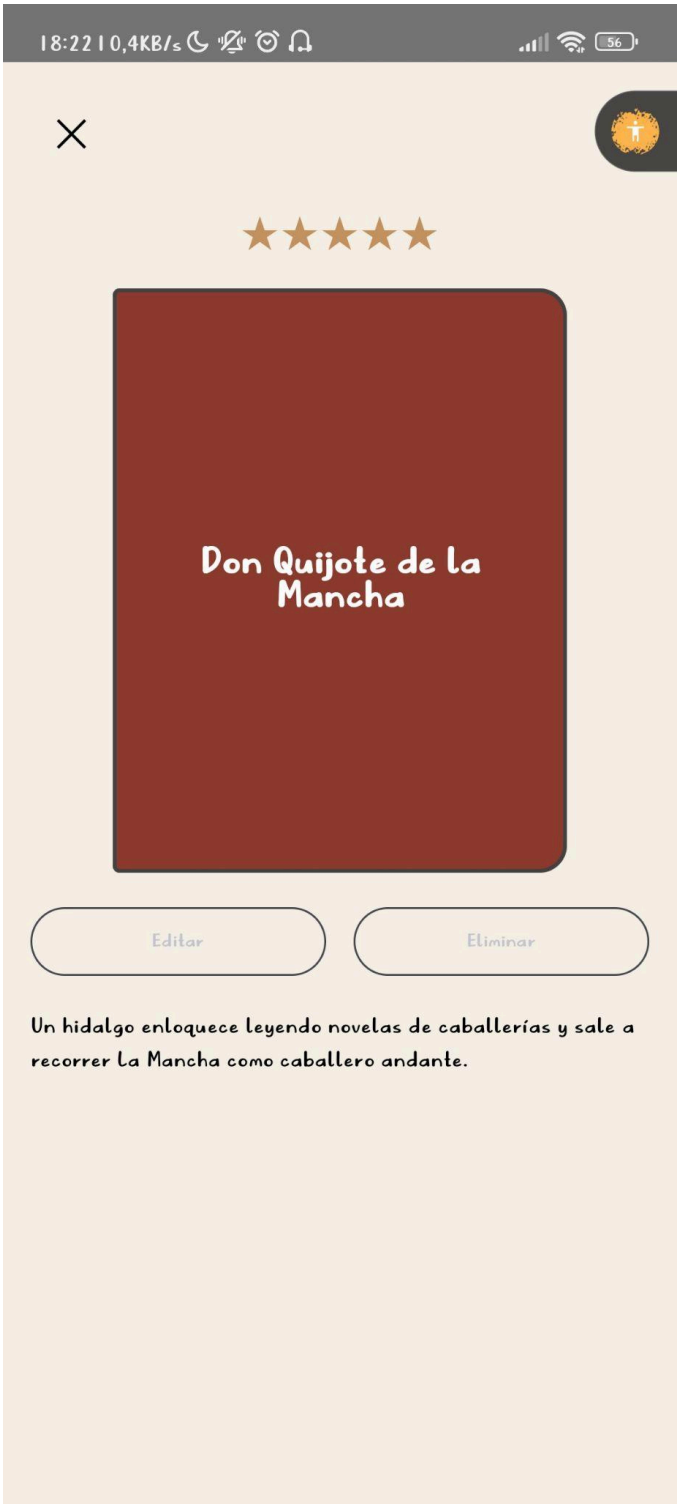
PantallaDetalle recibe el libro que se va a mostrar y onVolver, la lambda que avisa de que se quiere volver a la estantería. Enseña más información que la lista: el título completo, la valoración y la descripción.

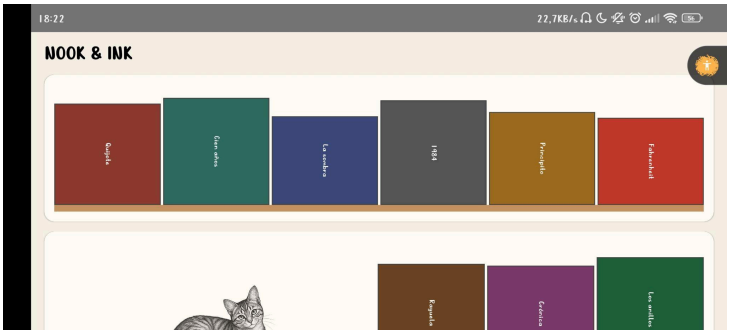
Con llenas = libro.valoracion.roundToInt() se redondea la nota para saber cuántas estrellas se dibujan llenas (4.6 da 5). El texto de las estrellas se forma cogiendo con take las primeras de «★★★★★» y completando hasta cinco con las de «☆☆☆☆☆». Ahora mismo lo hago así, pero en un futuro lo mejoraría para poder poner notas con decimales. formaDeLibro define las esquinas del libro: casi rectas a la izquierda, que es el lomo, y más redondeadas a la derecha, para simular la forma de un libro. Investigué cómo hacerlo en 3D y es posible, pero ahora mismo lo veo pronto para incorporar tanta lógica, así que lo haré en entregas posteriores.

La pantalla usa un Scaffold con innerPadding, igual que la estantería, y dentro una Column que ocupa toda la pantalla. Lleva verticalScroll para poder desplazarse si la descripción es larga o al girar el móvil. El botón X es un IconButton y al pulsarlo llama a onVolver. El libro grande es un Box con el color del libro y la forma de libro, con márgenes laterales (padding horizontal de 48 dp) para que no ocupe todo el ancho, y muestra el título en el centro, en blanco y en negrita. Debajo van los botones Editar y Eliminar, que son OutlinedButton con weight(1f) para ocupar la mitad cada uno, y todavía no hacen nada. Al final va la descripción del libro.

4.2 Capturas

Pantalla	Captura
Estantería (lista)	

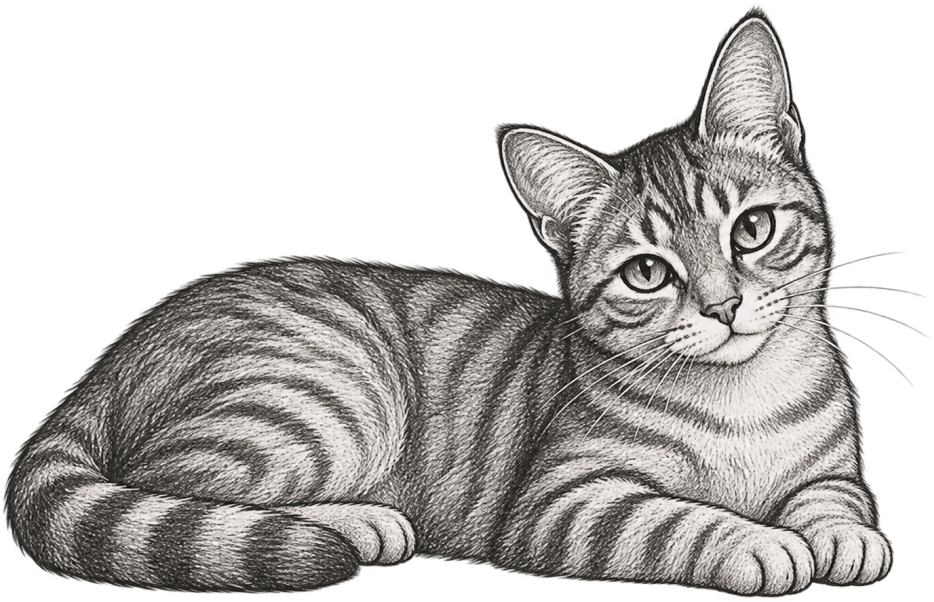
Detalle de un libro	

Estantería con el móvil girado	
Detalle con el móvil girado	

5. CONCLUSIONES Y VALORACIÓN PERSONAL

Esta entrega me ha servido para ver cómo se organiza una app con Compose: cada pantalla es una función que dibuja lo que recibe, y una pieza superior (AppPantallas) decide cuál se muestra según un estado.

Lo que más ha cambiado respecto a mi primera idea es la estructura, pensaba hacer una Activity por pantalla, pero al final, he visto que la forma recomendada para lo que viene después es una sola Activity con la navegación gestionada dentro de Compose



6. COSAS QUE NO HEMOS VISTO

Elemento	Dónde se usa	Para qué sirve	En los apuntes	
Modifier.rotate(90f)	LomoDeLibro (PantallaEstanteria.kt)	Gira el título 90 grados para escribirlo en vertical sobre el lomo.	No	Modifier.rotate
Modifier.wrapContentWidth(unbounded = true)	LomoDeLibro (PantallaEstanteria.kt)	Deja que el texto mida lo que necesite aunque el lomo sea estrecho, para que no se parta antes de girarlo.	No	wrapContentWidth
rememberLazyListState()	AppPantallas.kt y PantallaEstanteria.kt	Guarda la posición de la lista para que, al volver del detalle, no salte al principio.	No	Listas en Compose
BackHandler (opcional, comentado)	AppPantallas.kt	Hace que el botón atrás del móvil vuelva a la estantería en lugar de cerrar la app.	No	BackHandler

7. ANEXOS

Prototipo de pantallas

Ver imagen a tamaño completo

Video

Video

Github

Github